ReadRight 2.0 - Part 1: Evaluation Report

Milestone 1 - Evaluation & Project Selection

Mihir Patel & Mir Patel

Projects evaluated:

Candidate 3: ngmille/readright
Candidate 4: Jaredburne47/4150ReadRight
Candidate 5: ZhongTheKong/read_right_project

Project 1: ngmille/readright

Strengths

- The app has a clean, well-organized architecture. If Firebase fails to initialize, it automatically falls back to mock versions of the auth and data repositories so the app still runs.
- The practice screen handles edge cases well, including a permission-denied state with a direct link to Settings and a text-to-speech feature that reads the target word and example sentences aloud.

Weaknesses

- There is no real pronunciation scoring, pronunciation_service.dart only compares the transcript to the target word using string-matching, with no actual acoustic or phoneme-level analysis.
- The teacher dashboard is unfinished. The files exist but the screens are placeholders with no real class management, student review, or analytics.

Architecture

- State management uses Provider with role-based navigation, and the repository pattern makes it easy to swap implementations, adding new features doesn't require restructuring the whole app.
- The biggest limitation is that extending the speech pillar means building a brand-new cloud integration from scratch, since nothing real exists behind the current assessor interface.

User Experience

- Feedback states are clear and child-friendly and there is a visible countdown during recording, color-coded result cards, and a record button that disables itself during processing so kids can't accidentally double-tap.
- The app uses iOS-style (Cupertino) widgets throughout, which looks out of place on Android devices and Chromebooks which are the most common platforms in elementary classrooms.

Educational Effectiveness

- The app correctly tracks which words a student has mastered and advances them through the list, and the sample-sentence read-aloud reinforces words in context.
- Because scoring is based purely on what the speech-to-text transcribes rather than how the word was actually said, the feedback signal is unreliable.

Code Quality

- Naming conventions are consistent, methods are clearly named, and the data models are defensive and structured well.
- There is only one test file covering data models, and no widget tests, no service tests, and nothing that exercises the practice flow or the speech assessor.

Project 2 — Jaredburne47/4150ReadRight (readright_app):

Strengths

- This is the only repo with a fully working cloud pronunciation pipeline, it calls Microsoft's Azure Pronunciation Assessment API, parses the results properly, and blends in a similarity score so near-misses don't automatically fail.
- It has a working teacher dashboard, class and student management, CSV word-list upload, an animated mascot, and an accessibility toggle.

Weaknesses

- The README describes an offline fallback for when Azure is unavailable, but the actual fallback code (cloud_fallback_assessor.dart) just returns a zero score every time.
- The recording window is hardcoded to 3 seconds for every word, which will cut off longer words and leave unnecessary silence after short ones, and there are leftover debug print statements throughout the production code.

Architecture

The most directly extensible codebase for the three required pillars.

- Provider-based state management cleanly separates recording, scoring, and persistence into distinct services, making it straightforward to add new features without restructuring existing ones.
- The data models (ClassRoom, Student, AttemptRecord, AssessmentResult) are already shaped the right way for a teacher dashboard, a story builder, and games to be built on top of.

User Experience

Strong for both students and teachers, and the most complete of the three.

- The student practice screen has an animated mascot, color-coded feedback, and a star-burst success animation, which genuinely engaging for the target age group.
- Separate visual themes for students and teachers make it immediately clear which mode you're in, and the accessibility toggle (high-contrast, large-text) is the only persistent accessibility feature across all three repos.

Educational Effectiveness

The strongest educational foundation of the three.

- Real acoustic scoring measures how a word was actually said, not just whether the speech-to-text transcribed it correctly, a meaningfully better signal for a pronunciation app than Project 1's string-matching.
- Per-word accuracy data is already captured in the model but not yet shown to students in a useful way, which is a clear, concrete improvement opportunity for the speech pillar.

Code Quality

- Models are well-typed with backward-compatible JSON parsing, and the 9 dedicated test files are the best test coverage of the three repos.
- The main issues are removing debug print statements, fixing the fallback assessor, and making recording duration adaptive.

Project 3 — ZhongTheKong/read_right_project

Strengths

- Like Project 2, it uses real Azure-based pronunciation scoring rather than transcript string-matching, so the feedback is based on actual acoustic analysis.
- The recording screen shows a live progress bar tied to the actual recording timer, which is a small but useful UX touch that neither of the other two repos has.

Weaknesses

- All data is stored in one flat JSON file (all_user_data.json) that gets fully reloaded and rewritten every time anything changes, which won't scale and risks data loss if a save is interrupted.
- The app title still says "Navigation Demo" (leftover from the Flutter starter template) and widget tests are documented as broken in the README.

Architecture

- Three separate providers handle session state, recording, and user data cleanly, which is the right idea, but all of that data ultimately funnels into one file that gets rewritten as a whole.
- The README describes an architecture with separate folders and models that don't actually exist in the delivered code, which is a red flag about documentation reliability.

User Experience

- The live progress bar during recording is a plus, but there's no loading screen at startup, so the app can look frozen for a few seconds while it initializes.
- The Windows recording bug documented in the README is a direct risk for the milestone's required working-build demo if any team members are on Windows.

Educational Effectiveness

- Real Azure acoustic scoring is a genuine strength shared with Project 2 and gives students a more accurate picture of how they actually pronounced a word.
- It doesn't handle empty results gracefully or blend in a similarity score for near-misses, making it more likely to mark a borderline attempt as flat-out wrong.

Code Quality

- The PRE/POST comment convention on service methods is clear and consistent.
- Mock test classes are defined inside production code, the default Flutter boilerplate test is still in the test suite, and the known Windows bug is unresolved.